1. Use Case Diagram Overview

- Actors:
    - User (Student / Grader / System Operator): External entity that executes the program and provides input puzzle files.
- System Boundary: Sudoku Solver System
- Primary Use Case: Solve Sudoku Puzzles

2. Use Case Specification

a. Identify / Discover the Use Case

- Use Case Name: Solve Sudoku Puzzles
- Actor: User
- Description of Discovery: Discovered by analyzing the core functional requirement of the application—taking partially completed 9×9 Sudoku puzzle text files, processing their contents, executing an exhaustive depth-first search (DFS) with backtracking, and outputting all valid solutions or a no-solution notice.

b. Brief Description for the Use Case

The Solve Sudoku Puzzles use case allows a user to provide text files containing 9×9 Sudoku puzzles with blank cells represented by underscores (_). The system parses the files, validates the board constraints, applies a recursive DFS backtracking algorithm to discover all valid grid completions, and prints the file name, initial puzzle state, and solved configurations to the console.

c. Step-by-Step Flow (Outline of Events) for the Use Case

1. Initiation: The user launches the compiled executable (Assignment4.exe) via the command line, passing puzzle file paths as arguments (e.g., puzzle1.txt), or enters them interactively if no arguments are provided.
2. File Loading: The system iterates through the provided file list, opening each text stream and reading character data.
3. Parsing & Grid Initialization: The system filters out whitespace, extracts exactly 81 valid characters (1–9 and _), and populates the working board and initial board arrays.
4. Display Initial State: The system outputs a visual banner with the file name and renders the initial puzzle board grid with 3x3 subgrid formatting.
5. Execution (DFS Backtracking): The system invokes the solver, which recursively scans for empty cells, places candidate digits while verifying row, column, and 3x3 block constraints, and backtracks upon hitting conflicts.
6. Output Generation:
    - If solutions are found, the system prints each valid solution sequentially with a solution count header.
    - If no solution exists, the system outputs "No solution found".
7. Termination: The program concludes processing once all input files have been evaluated.

d. Detailed Description for the Use Case

- Pre-conditions:
    - The program must be compiled successfully into Assignment4.exe and executed within the designated grading environment (EECS Cycle servers or lab machines).
    - Valid puzzle text files (puzzle1.txt through puzzle5.txt) must exist in the working directory.
- Post-conditions:
    - All valid solutions for each input puzzle are printed to standard output.
    - Resources and object memory are safely released upon completion of each file iteration.
- Normal Flow:
    - User initiates the program with valid puzzle files.
    - System loads each file, validates board dimensions, displays the initial grid, computes solutions via DFS backtracking, and outputs the results.
- Alternative / Exception Flows:
    - *File Access Failure:* If a file cannot be opened, the system writes an error message to stderr and safely skips to the next file.
    - *Invalid File Format:* If a file contains an incorrect number of cells or invalid characters, the system flags a loading failure.
    - *Unsolvable Puzzle:* If a puzzle configuration yields no valid solutions under Sudoku rules, the system completes the search and correctly prints "No solution found".